CHANDIGARH UNIVERSITY

Department of Computer Applications

PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM

A C++ Object-Oriented Programming Project

Submitted by

NITIN RAJ

UID: 25BCA10383

BCA – 3rd Semester
Subject: Object-Oriented Programming using C++

Submitted to: Department of Computer Applications
Chandigarh University

CHANDIGARH UNIVERSITY

Department of Computer Applications

CERTIFICATE

This is to certify that the project report entitled “Student Record Management System” is submitted by Nitin Raj (UID: 25BCA10383), student of BCA 3rd Semester, Department of Computer Applications, Chandigarh University, as part of the academic work for Object-Oriented Programming using C++. The project demonstrates basic object-oriented programming concepts through a menu-driven C++ application. The work is prepared for academic learning and evaluation under the guidance of the concerned faculty member.

Faculty Signature: _____
Name of Faculty: _____
Date: _____
College Seal: _____

DECLARATION

I, Nitin Raj, UID 25BCA10383, student of BCA 3rd Semester, hereby declare that the project entitled “Student Record Management System” has been prepared by me for academic purposes as part of the Object-Oriented Programming using C++ subject. The project is based on my understanding of the subject and demonstrates the use of classes, objects, constructors, encapsulation and member functions. The program and report have been prepared for learning and demonstration. Books, websites and learning resources consulted for understanding C++ are listed in the bibliography.

Student Signature: _____
Name: Nitin Raj
UID: 25BCA10383
Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject teacher for providing guidance, support and encouragement during the preparation of this project entitled “Student Record Management System using C++.” The explanations and practical learning of object-oriented programming helped me understand how a real-life information-management task can be represented through classes and objects. I am thankful to Chandigarh University and the Department of Computer Applications for providing the academic environment and opportunity to complete this project as part of my BCA third-semester coursework. Classroom lectures, practical sessions and study materials helped me understand the concepts required for designing and developing the application. I would also like to thank my classmates and friends for their suggestions and cooperation during the project work. I am grateful to my family for their motivation and continuous support. Finally, I acknowledge the developers of the C++ language, compiler tools and online learning resources that helped me practise, compile and understand the program. This project has been a useful learning experience, and I thank everyone who contributed directly or indirectly to its completion.

ABSTRACT

The Student Record Management System is a menu-driven application developed using the C++ programming language to store, display and search basic student information. The main purpose of this project is to demonstrate how a computer program can organize student details in a systematic manner and how Object-Oriented Programming concepts can be applied to a familiar academic task. The application accepts information such as roll number, student name, course, semester and marks. It creates a student record and displays the information in a structured format.

The project uses a class named Student to combine student data and the functions that operate on that data. Private data members help protect the information from direct access, while public member functions provide controlled operations. Default and parameterized constructors are included to demonstrate object initialization. A vector stores multiple Student objects during program execution. The menu allows the user to add a student record, display all available records, search for a student by roll number and exit the application.

The project is suitable for a beginner-level BCA student because it focuses on fundamental C++ and OOP concepts rather than complex external libraries or database connectivity. It is a console-based academic model. Records are stored temporarily in memory and are lost when the program closes. The project provides a foundation for future improvements such as file handling, database integration, record updating and deletion.

Keywords: C++, OOP, Student Management, Class, Object, Encapsulation, Constructor, Vector, Student Record.

TABLE OF CONTENTS

1. Introduction and Background

2. Problem Statement

3. Objectives and Scope

4. Existing and Proposed System

5. Requirement Analysis

6. OOP Concepts Used

7. System Design and Architecture

8. Algorithm and Flowchart

9. Class Design

10. Program Modules

11. C++ Source Code – Part 1 to 4

12. Sample Output

13. Testing Strategy and Test Cases

14. Advantages, Limitations and Future Scope

15. Conclusion and Learning Outcomes

16. Bibliography and Viva Questions

1. INTRODUCTION AND BACKGROUND

Student information is an important part of the daily work of schools, colleges and other educational institutions. Basic details such as a student's roll number, name, course, semester and marks are needed for identification and academic record keeping. When these details are maintained only in notebooks or separate paper files, searching for a particular record may take time. Information may also be written in different formats, making it difficult to maintain consistency. A simple computer application can organize such information and make basic record operations easier to perform.

The Student Record Management System is developed as a beginner-level C++ project to demonstrate how a familiar academic task can be represented through a program. It provides a menu through which a user can add a student record, display all records, search for a student using a roll number and exit the application. Each record contains a roll number, name, course, semester and marks. The program displays the information in a structured format so that it can be read easily.

The project uses Object-Oriented Programming concepts. A class named Student combines the data members and functions related to a student. Objects are created from this class, and a vector stores multiple objects while the program is running. Constructors initialize the objects, while public member functions display information and return the roll number needed for searching.

The application is intentionally limited to a console-based academic demonstration. It does not use a database or permanent file storage. Its main value is to help a BCA student understand how classes, objects, encapsulation and basic C++ programming can be applied to a practical example.

2. PROBLEM STATEMENT

Educational institutions need to maintain information about students for academic and administrative purposes. In a small classroom or sample record system, details may be written manually in registers or stored in separate documents. This method can be sufficient for a limited number of entries, but it may become inconvenient when records increase. A person may need to search several pages to find one student, rewrite information when preparing a list, or check details repeatedly. Manual handling can also lead to spelling mistakes, missing fields and inconsistent presentation.

A basic digital system can help organize student details in a common format. However, a beginner programming project should remain simple enough to understand and demonstrate. The problem selected for this project is therefore limited to adding student details, displaying the records entered during the current session and searching for a student by roll number. The system accepts a roll number, name, course, semester and marks. It checks selected values, creates a Student object and stores it in a vector.

The proposed application does not attempt to manage admissions, attendance, fee payments, examination schedules or official academic results. It is not connected to an institution's database and does not provide multiple user accounts. These limits are important because the project is intended to demonstrate basic C++ OOP concepts rather than operate as a complete college information system.

The project provides a simple and consistent method for handling sample student records. It also gives the student developer an opportunity to practise class design, constructors, encapsulation, loops, conditions, vectors and formatted output.

3. OBJECTIVES AND SCOPE

The main objective of this project is to develop a simple Student Record Management System using C++ and apply fundamental Object-Oriented Programming concepts to a practical academic example. The project aims to organize student details in one structure and provide a menu-based method to perform common record operations. It also aims to improve the student's understanding of how a real-world entity can be represented as a class and how multiple objects can be stored and processed.

The specific objectives are to create a Student class, declare private data members, initialize objects through constructors and provide public member functions. The application should accept a roll number, student name, course, semester and marks. It should allow a user to add a record, display all records entered in the current session and search for a particular student using the roll number. Another objective is to use a vector to store a flexible number of Student objects and to format the displayed information in a readable way.

The scope of the current version is limited to basic student record operations in a console environment. It can be used for classroom demonstrations and small sample datasets. The marks field accepts values from 0 to 100, and the roll number and semester must be positive values. The system does not permanently save information. All objects remain in memory only until the program is closed.

Features such as editing or deleting records, calculating grades, attendance tracking, fee management, login security, database connectivity and report printing are outside the present scope. These features may be added in future versions after the basic application is understood and tested.

4. EXISTING AND PROPOSED SYSTEM

In a manual student record system, information is commonly maintained in registers, paper forms or separate spreadsheet files. A person responsible for the records writes the student's roll number, name, course, semester and marks. When a particular entry is required, the person searches the register or opens the relevant file. This approach is familiar and does not require programming knowledge, but it may become slow when the number of records increases. Repeated writing can create spelling errors, missing information and differences in the way records are presented.

The proposed Student Record Management System is a small C++ application that provides a common menu for basic record operations. The user can enter a student's details and the program creates an object of the Student class. The object is stored in a vector. The user can display all stored records or search for a student by roll number. A single display function presents the fields in a consistent format. This makes the program easier to demonstrate and helps reduce repeated manual formatting.

The proposed system is technically feasible because it uses standard C++ features and can be compiled with a commonly available compiler. It is economically feasible because no paid software or special hardware is required. It is operationally feasible for a beginner because the menu contains a small number of clearly named options.

The proposed application is not a complete replacement for a college management platform. It does not provide permanent storage, user authentication or secure handling of official student data. It is an educational prototype intended to demonstrate basic record management and object-oriented program design.

5. REQUIREMENT ANALYSIS

Requirement analysis identifies the hardware, software and functions needed to develop and run the application. The hardware requirements are modest. A desktop computer or laptop with a keyboard, monitor and sufficient memory to run an operating system and a C++ development environment is adequate. No dedicated server, network connection or special peripheral device is required. A printer may be used separately if the student wishes to print the report or capture the output for submission.

The software requirements include an operating system such as Windows, Linux or macOS and a C++ compiler that supports the language features used in the source code. The program is suitable for C++11 or later and can be compiled using a C++17 setting. An editor or integrated development environment such as Visual Studio Code, Code::Blocks, Dev-C++ or Visual Studio may be used. The program uses standard headers including iostream, iomanip, vector, string and limits.

The functional requirements describe the operations the application must perform. It must accept student roll number, name, course, semester and marks. It must create and store a record, display all records in the current session and search for a record using its roll number. It must provide a menu and an exit option. It should also reject selected invalid values such as a non-positive roll number, non-positive semester or marks outside the range of zero to one hundred.

The non-functional requirements include usability, readability, reliability of the basic operations and maintainability of the code. The menu should be easy to understand, the output should be clearly formatted and the functions should have separate responsibilities. Since the program stores data only in memory, permanent record security is not part of this version.

6. OOP CONCEPTS USED

Object-Oriented Programming is a programming approach in which data and the functions that operate on that data are organized into objects. This project uses a class named Student to represent one student's record. The class defines the information that every student object contains and the operations that can be performed on that information. Each student entered by the user becomes an individual object created from the Student class.

Encapsulation is demonstrated by declaring the data members under the private access specifier. The roll number, name, course, semester and marks cannot be accessed directly from the main function. Instead, the constructors initialize these values and public member functions provide the required operations. The getRollNumber() function returns the roll number for searching, while displayRecord() prints the student's information. This arrangement keeps data and related functions together and limits direct access to the internal data.

Constructors are special member functions that are called when an object is created. The default constructor assigns safe initial values. The parameterized constructor accepts the actual values entered by the user and assigns them to the object's data members. Member functions perform tasks related to the object and make the class easier to use.

The program also uses a vector<Student> to store multiple objects. A vector can grow as new records are added, so the program does not need a fixed number of student entries. Loops and conditional statements control the menu and search operations. Together, these features demonstrate classes, objects, encapsulation, constructors, member functions and object collections in a practical C++ application.

7. SYSTEM DESIGN AND ARCHITECTURE

The application follows a simple console-based architecture. The user interaction and menu control are handled by the main() function. The Student class forms the data and operation part of the program. It stores the details of a student and provides functions to display the record and return the roll number. The vector in main() stores the objects that are created when users add new student entries.

When the program starts, an empty vector named students is created. The menu is displayed and the user selects an operation. If the user chooses to add a record, the program accepts the required details and checks selected values. If the values are valid, the parameterized constructor creates a Student object and emplace_back() adds it to the vector. The displayRecord() function is then called to show the newly added record.

If the user selects display-all, the program checks whether the vector is empty. If it contains records, a range-based for loop visits each object and displays it. If the user selects search, the program accepts a roll number and compares it with the value returned by getRollNumber() for each object. A matching record is displayed; otherwise, a not-found message is shown.

This design separates the responsibility of storing and displaying student information from the responsibility of controlling the menu. It makes the source code easier to read and explain. All processing takes place locally, and no server or database is contacted. The vector is temporary, so its contents are removed when the program ends. A future version could add a separate storage layer and split the code into multiple files.

8. ALGORITHM AND FLOWCHART

An algorithm is a sequence of steps used to solve a problem. The algorithm of this project begins by starting the program and creating an empty vector to store Student objects. The system displays the main menu and asks the user to enter a choice. The menu is repeated until the user selects the exit option. This loop allows several records to be entered, displayed or searched during one execution.

When the user selects the add-record option, the program asks for the roll number, student name, course, semester and marks. It checks that the roll number and semester are positive and that marks are between zero and one hundred. If the entered values are invalid, an error message is displayed. Otherwise, the parameterized constructor is used to create a Student object, and the object is added to the vector. The record is then displayed.

When the user selects display-all, the program checks whether the vector contains any objects. If it is empty, a suitable message is printed. If records are available, a loop displays each object. When search is selected, the user enters a roll number. The program compares this number with the roll number of every stored object. If a match is found, the corresponding record is displayed. If no match is found, a not-found message appears.

If the user selects exit, the program displays a closing message and ends the loop. Any other menu choice produces an invalid-choice message. A flowchart for this algorithm would show Start, Display Menu, Read Choice, decision branches for the four options, the related operation and a return arrow to the menu. This representation makes the program sequence easier to understand before reading the source code.

9. CLASS DESIGN

The main class used in the project is named Student. It represents one student and stores the information required for a basic student record. The class groups related data and operations in one place. This design is suitable for an object-oriented application because every student record follows the same structure, while each object contains different values.

The private section contains five data members: rollNumber, name, course, semester and marks. The rollNumber variable stores the unique identifying number entered for the student. The name and course variables are strings because they may contain more than one word. The semester variable stores the current semester number, and marks stores the student's marks as a decimal value. These variables are private so that the main function cannot change them directly.

The public section contains a default constructor and a parameterized constructor. The default constructor initializes the data members with safe starting values. The parameterized constructor receives the actual details and assigns them to the object. The displayRecord() member function prints all details in a structured format. It uses output manipulators such as setw(), left, fixed and setprecision() to align the labels and show marks with two decimal places.

The getRollNumber() function returns the roll number. It is used by the search operation to compare the entered roll number with each stored object. The class therefore provides a small and clear public interface. The design can be extended in future versions with functions to update marks, calculate grades, edit details or validate duplicate roll numbers.

10. PROGRAM MODULES

The application is divided into logical modules, even though the complete source code is written in one C++ file. Each module has a separate responsibility. The first module is student initialization. The default constructor assigns initial values to a Student object, while the parameterized constructor receives the details entered by the user. This ensures that the object is initialized when it is created.

The second module is record display. The displayRecord() function prints the roll number, name, course, semester and marks. It uses the iomanip library to align the output and display marks in a consistent numeric format. Keeping display logic inside the class avoids repeating the same output statements in different menu branches.

The third module is record storage. A vector<Student> named students stores all objects added by the user. The vector is flexible because it can hold a changing number of records. The emplace_back() operation constructs a new Student object at the end of the vector using the supplied values.

The fourth module is menu management. The main() function displays the available choices and uses a do-while loop to repeat the menu until exit is selected. Conditional statements decide which operation should run. The fifth module is searching. It reads a roll number and checks each object by calling getRollNumber(). If a match is found, displayRecord() is called; otherwise, a message is shown.

The final module is basic input checking. The program checks selected numeric values and handles non-numeric menu input. These modules make the program easier to explain, test and improve.

11. C++ SOURCE CODE – PART 1

Part 1 of the source code introduces the required header files and declares the Student class. The iostream header is used for keyboard input and screen output through cin and cout. The iomanip header provides formatting tools such as setw(), fixed and setprecision(). The vector header allows the program to store a flexible collection of Student objects. The string header supports student names and course names that may contain spaces. The limits header is used to clear invalid input from the input stream.

The class declaration begins with the class keyword followed by the name Student. The private section declares the data members rollNumber, name, course, semester and marks. These values describe one student record. They are private to demonstrate encapsulation and prevent the main function from directly changing the internal data.

The public section begins with the default constructor. A constructor is called automatically when an object is created. The default constructor assigns safe initial values such as zero and “Not Set”. The parameterized constructor accepts five values: roll number, student name, course, semester and marks. It assigns each parameter to the corresponding data member. This allows a Student object to be created with all its details in one operation.

This part establishes the structure used by the rest of the application. When studying it, identify the difference between data members and member functions, and understand why private and public access sections are used. The class declaration is the foundation for the display and search operations that appear in later parts.

12. C++ SOURCE CODE – PART 2

Part 2 of the source code contains the member functions of the Student class. The displayRecord() function prints a formatted student record. It first prints a heading and separator lines, then displays the roll number, name, course, semester and marks. The output manipulators help align the labels and values. The fixed and setprecision(2) manipulators display marks with two digits after the decimal point, making the record look clear and consistent.

The getRollNumber() function returns the value stored in the private rollNumber data member. It is declared as a public function so that the main function can search for a student without directly accessing private data. Both displayRecord() and getRollNumber() are declared const. This indicates that these functions do not modify the object's stored information.

After the class definition, main() begins. A vector named students is declared to store Student objects. The integer variable choice stores the menu selection. A do-while loop is used because the menu should appear at least once and continue until the user selects the exit option. The menu contains options to add a student record, display all records, search by roll number and exit.

The program attempts to read the user's choice. If the input is not numeric, the input stream is cleared and the remaining invalid characters are ignored. The loop then returns to the menu. This part connects the Student class with the menu-driven interface and demonstrates how member functions can be called from main().

13. C++ SOURCE CODE – PART 3

Part 3 explains the code used to add and display student records. When the user selects option one, the program declares variables to receive the roll number, semester, name, course and marks. It first reads the roll number. The input buffer is cleared before using getline() so that the full student name can be accepted, including spaces. The course name is also read using getline() for the same reason.

The program then asks for the semester and marks. It checks whether the roll number is greater than zero, whether the semester is positive and whether marks fall between zero and one hundred. If any of these conditions is not satisfied, the program displays an invalid-details message. This is a basic form of input validation and helps prevent obviously incorrect sample records.

If the values are acceptable, the statement students.emplace_back() creates a Student object at the end of the vector. The parameterized constructor receives the entered values. A confirmation message is printed, and students.back().displayRecord() displays the record that was just added. The back() function provides access to the last object in the vector.

When option two is selected, the program checks whether the vector is empty. If there are no records, it displays a message. Otherwise, a range-based for loop visits each Student object and calls displayRecord(). This part demonstrates conditional statements, object creation, vector storage, constructors and loops in a practical context.

14. C++ SOURCE CODE – PART 4

Part 4 contains the search operation, exit operation and the closing statements of the program. When the user selects option three, the program declares a variable named searchRoll and a Boolean variable named found. The user is asked to enter the roll number that needs to be searched. The program then uses a range-based for loop to visit every Student object stored in the vector.

For each object, getRollNumber() is called to obtain its roll number. The returned value is compared with the number entered by the user. If both values are equal, the program calls displayRecord() for that object, sets found to true and stops searching by using break. If the loop finishes without finding a match, the value of found remains false and the program displays “Student record not found.”

When option four is selected, a thank-you message is displayed. The do-while loop ends because the condition checks whether choice is not equal to four. If the user enters any other number outside the available menu options, the program displays an invalid-choice message and returns to the menu.

The final return 0 statement indicates that the program has completed successfully. This part demonstrates searching through a vector, Boolean variables, comparison operators, loops, conditional statements and program termination. Together with the earlier parts, it completes the menu-driven Student Record Management System.

15. SAMPLE OUTPUT

The sample output demonstrates how the user interacts with the Student Record Management System after compiling and running the program. When the application starts, it displays a menu containing four options. The user selects option one to add a student record. The program asks for the roll number, student name, course, semester and marks. For example, the user may enter roll number 103, name Nitin Raj, course BCA, semester 3 and marks 82.50.

After the values are entered, the program checks the selected conditions. Since the roll number and semester are positive and the marks are within the range of zero to one hundred, the program accepts the details. A Student object is created using the parameterized constructor and added to the vector. The program displays a confirmation message and prints the new student record.

If the user selects option two, the program displays every student record stored in the vector. If the user selects option three, the program asks for a roll number. When the entered number matches a stored record, the corresponding details are displayed. If no matching record exists, the program displays a not-found message. Option four closes the application.

The output is formatted using iomanip functions so that the labels are aligned and marks are displayed with two decimal places. This sample is intended to show the expected behaviour of the program. The exact spacing may differ slightly depending on the terminal or IDE used to run the source code.

16. TESTING STRATEGY AND TEST CASES

Testing is the process of checking whether a program performs the required operations correctly. The Student Record Management System should be tested with valid information, invalid values and different menu choices. Testing helps identify errors in input handling, object creation, record storage, display and searching. The source file should first be compiled successfully, and then each menu option should be tried with suitable sample data.

A valid-entry test can use roll number 103, name Nitin Raj, course BCA, semester 3 and marks 82.50. The expected result is that the record is accepted and displayed. A display-all test should be performed after adding two or more students. The expected result is that each stored object is displayed. A search test using an existing roll number should display the matching student. A search test using a roll number that is not stored should display the not-found message.

An empty-list test selects display-all before any student has been added. The expected result is a message that no student records are available. An invalid-data test enters a zero or negative roll number, a non-positive semester or marks below zero or above one hundred. The current program should reject these values. An invalid-menu test enters a number outside the range one to four and checks that the menu displays an appropriate message.

These tests cover the main features of the application. They do not guarantee that every possible input problem has been handled. For example, additional validation would be needed for non-numeric marks, duplicate roll numbers and empty names. The results should be recorded during the actual demonstration and compared with the expected behaviour.

17. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

The Student Record Management System has several advantages as a beginner-level programming project. It provides a simple menu that guides the user through the available operations. Student details are stored in a common structure, and the display function presents the information in a consistent format. The Student class demonstrates how data and related functions can be grouped together. Constructors initialize objects, while a vector allows the program to handle a flexible number of student records during execution. The search option also demonstrates how objects can be checked using a data member returned through a public function.

The project has limitations. Records are stored only in memory and disappear when the program closes. There is no login facility, password protection or role-based access. The application does not connect to a college database and cannot retrieve official records. It does not include options to edit or delete an existing student record. It also does not calculate grades, attendance, fees or examination eligibility. Input validation is basic and duplicate roll numbers are not prevented in the current version.

Several improvements can be made in future versions. File handling or database connectivity can be added to save records permanently. Update and delete options can help users correct or remove records. A grade-calculation module could convert marks into grades according to a defined academic policy. Login security could be added for controlled access. The project could also be extended with attendance management, report generation and a graphical user interface. These improvements would make the system more useful while retaining the basic object-oriented design.

18. CONCLUSION AND LEARNING OUTCOMES

The Student Record Management System demonstrates how a basic academic record task can be represented through a C++ object-oriented application. The program accepts student details, creates objects from the Student class and stores them in a vector during execution. It provides menu options to add a record, display all available records and search for a student using a roll number. The display function presents the information in a structured format, making the result easy to read during a classroom demonstration.

The project provides practical experience with important OOP concepts. The Student class represents a real-world entity, while each object stores the details of one student. Encapsulation is applied by declaring data members private and providing public functions for controlled access. The default and parameterized constructors demonstrate object initialization. Member functions perform record display and return the roll number required for searching.

The project also provides practice with C++ input and output, strings, vectors, loops, conditional statements and output formatting. It helps the student understand how to compile a source file, enter test data, check program behaviour and document the design of an application. These skills are useful for later programming subjects and larger software development tasks.

The current version is an educational prototype and should not be used as an official institutional database. Permanent storage, stronger validation, security and additional student services can be added in future versions. Overall, the project meets its academic purpose by connecting fundamental C++ OOP concepts with a familiar student information-management problem.

19. BIBLIOGRAPHY AND VIVA QUESTIONS

The bibliography identifies the books, documentation and learning materials that support the understanding and development of this project. Bjarne Stroustrup's The C++ Programming Language provides detailed information about the C++ language and its design. Herbert Schildt's C++: The Complete Reference explains core language features with examples. The cppreference website is a useful online reference for C++ syntax and standard library components such as vector, string, iostream and iomanip. The course syllabus and classroom notes for Object-Oriented Programming using C++ are also relevant because they identify the concepts expected in the academic project.

Viva preparation is important because a student should be able to explain the program and its working, not only submit the printed report. The student should be ready to define a class and an object, explain encapsulation and identify the private and public members of the Student class. The purpose of the default and parameterized constructors should also be explained. The student should understand why vector is used and how the search operation compares roll numbers.

Possible viva questions include: What is a class? What is an object? What is encapsulation? Why are data members private? What is a constructor? What is the difference between a default and parameterized constructor? Why is getline() used for names? What is a vector? How does the search function work? What happens when the vector is empty? What are the limitations of this project?

During a demonstration, the student can compile the program, add a sample record, display all records and search for an existing and non-existing roll number. Explaining the limitations and future improvements shows an understanding of the project's scope.

20. C++ SOURCE CODE – PART 1

This section contains part 1 of the complete C++ source code for the Student Record Management System. The code is divided only for report readability. All parts together form one source file and should be compiled as a single program.

```
001  #include <iostream>
002  #include <iomanip>
003  #include <vector>
004  #include <string>
005  #include <limits>
006  using namespace std;
007
008  class Student {
009  private:
010      int rollNumber;
011      string name, course;
012      int semester;
013      double marks;
014
015  public:
016      Student() {
017          rollNumber = 0; name = "Not Set"; course = "Not Set";
018          semester = 0; marks = 0.0;
019      }
020
021      Student(int roll, string studentName, string studentCourse,
022              int sem, double studentMarks) {
023          rollNumber = roll; name = studentName; course = studentCourse;
024          semester = sem; marks = studentMarks;
025      }
026
027      void displayRecord() const {
028          cout << "\\n=======================================\\n";
029          cout << "             STUDENT RECORD\\n";
030          cout << "=======================================\\n";
031          cout << left << setw(20) << "Roll Number:" << rollNumber << '\n';
032          cout << left << setw(20) << "Name:" << name << '\n';
033          cout << left << setw(20) << "Course:" << course << '\n';
034          cout << left << setw(20) << "Semester:" << semester << '\n';
```

20. C++ SOURCE CODE – PART 2

This section contains part 2 of the complete C++ source code for the Student Record Management System. The code is divided only for report readability. All parts together form one source file and should be compiled as a single program.

```
035          cout << fixed << setprecision(2);
036          cout << left << setw(20) << "Marks:" << marks << '\n';
037          cout << "=======================================\\n";
038      }
039
040      int getRollNumber() const { return rollNumber; }
041  };
042
043  int main() {
044      vector<Student> students;
045      int choice;
046      do {
047          cout << "\\n====== STUDENT RECORD MANAGEMENT SYSTEM ======\\n";
048          cout << "1. Add student record\\n2. Display all records\\n";
049          cout << "3. Search student by roll number\\n4. Exit\\n";
050          cout << "Enter your choice: ";
051          if (!(cin >> choice)) {
052              cout << "Invalid input. Enter a number.\\n";
053              cin.clear();
054              cin.ignore(numeric_limits<streamsize>::max(), '\n');
055              continue;
056          }
057          if (choice == 1) {
058              int roll, semester; string name, course; double marks;
059              cout << "Enter roll number: "; cin >> roll;
060              cin.ignore(numeric_limits<streamsize>::max(), '\n');
061              cout << "Enter student name: "; getline(cin, name);
062              cout << "Enter course: "; getline(cin, course);
063              cout << "Enter semester: "; cin >> semester;
064              cout << "Enter marks (0 to 100): "; cin >> marks;
065              if (roll <= 0 || semester <= 0 || marks < 0 || marks > 100) {
066                  cout << "Invalid details. Please check the values.\\n";
067              } else {
068                  students.emplace_back(roll, name, course, semester, marks);
```

20. C++ SOURCE CODE – PART 3

This section contains part 3 of the complete C++ source code for the Student Record Management System. The code is divided only for report readability. All parts together form one source file and should be compiled as a single program.

```
                cout << "Student record added successfully.\\n";
                students.back().displayRecord();
            }
        } else if (choice == 2) {
            if (students.empty()) cout << "No student records available.\\n";
            else for (const Student &student : students) student.displayRecord();
        } else if (choice == 3) {
            int searchRoll; bool found = false;
            cout << "Enter roll number to search: "; cin >> searchRoll;
            for (const Student &student : students) {
                if (student.getRollNumber() == searchRoll) {
                    student.displayRecord(); found = true; break;
                }
            }
            if (!found) cout << "Student record not found.\\n";
        } else if (choice == 4) {
            cout << "Thank you for using Student Record Management System.\\n";
        } else cout << "Invalid choice. Select 1 to 4.\\n";
    } while (choice != 4);
    return 0;
}
```

20. C++ SOURCE CODE – PART 4

This section contains part 4 of the complete C++ source code for the Student Record Management System. The code is divided only for report readability. All parts together form one source file and should be compiled as a single program.

21. SAMPLE CONSOLE SESSION

Illustrative input: roll number 103; name Nitin Raj; course BCA; semester 3; marks 82.50.

```
====== STUDENT RECORD MANAGEMENT SYSTEM ======

1. Add student record

2. Display all records

3. Search student by roll number

4. Exit

Enter your choice: 1

Enter roll number: 103

Enter student name: Nitin Raj

Enter course: BCA

Enter semester: 3

Enter marks (0 to 100): 82.50

Student record added successfully.

             STUDENT RECORD

Roll Number:        103

Name:               Nitin Raj

Course:             BCA

Semester:           3

Marks:              82.50
```

The displayed values are an illustrative sample. The program will show the details entered by the user.

22. COMPILATION AND EXECUTION

Save the source code as Student_Record_Management_System.cpp. Open a terminal or command prompt in the folder containing the file. Compile the program with a C++ compiler that supports C++11 or later. A sample command using g++ is: g++ -std=c++17 Student_Record_Management_System.cpp -o student_record. On Linux or macOS, run it using ./student_record. On Windows, run student_record.exe. In an IDE, create or open a C++ source file, paste the complete code, select a suitable language standard, build the project and run it. Follow the menu prompts to add, display and search student records.

23. TEST CASE SUMMARY

Valid student details | Record should be added and displayed | Expected: Success

Display records after adding entries | All stored records should appear | Expected: Success

Search using an existing roll number | Matching record should appear | Expected: Success

Search using an unknown roll number | Not-found message should appear | Expected: Success

Display when no records exist | Empty-record message should appear | Expected: Success

Enter invalid marks below 0 or above 100 | Record should be rejected | Expected: Success

Enter an invalid menu option | Invalid-choice message should appear | Expected: Success

24. GLOSSARY

Class: A user-defined type that groups data members and member functions.

Object: An instance of a class.

Encapsulation: Combining data and functions while controlling access to internal data.

Constructor: A special member function used to initialize an object.

Vector: A dynamic sequence container provided by the C++ Standard Library.

Roll Number: A number used to identify a student record in this project.

Member Function: A function declared inside a class.

Private: An access specifier that restricts direct access from outside the class.

Public: An access specifier that allows access through the class interface.

25. PROJECT REVIEW

Project Title: Student Record Management System
Student Name: Nitin Raj
UID: 25BCA10383
Course: BCA – 3rd Semester
Subject: Object-Oriented Programming using C++
Programming Language: C++
Project Type: Console-based academic application
Core Concepts: Class, Object, Encapsulation, Constructors, Vector

Faculty Remarks:

_____
_____
_____

Marks / Grade: _____
Faculty Signature: _____
Date: _____